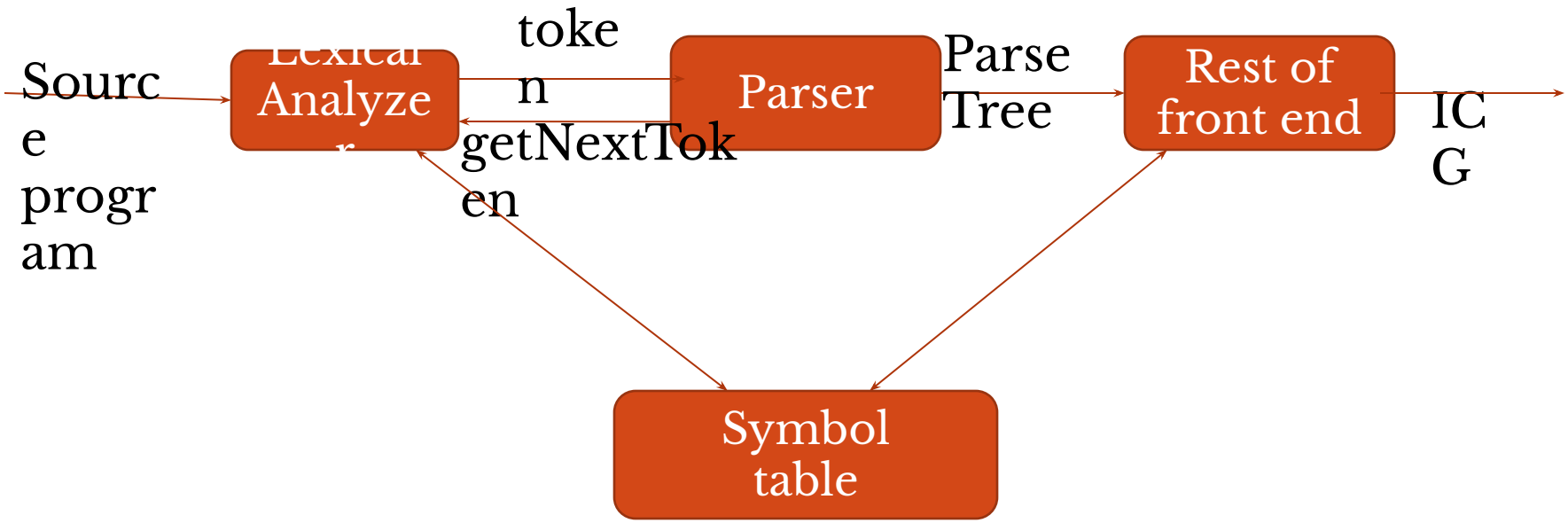
UNIT III

SYNTAX ANALYSIS

Topics

- Role of the parser
- Writing Grammars
- Context-Free Grammars
- Top Down parsing
  - Recursive Descent Parsing
  - Predictive Parsing
- Bottom-up parsing
  - Shift Reduce Parsing
  - Operator Precedence Parsing
  - LR Parsers
    - SLR Parser
    - Canonical LRParser
    - LALR Parser

The role of Parser



Parsing Techniques

- There are two types of parsing
  - **Top Down parsing**
    - **Recursive Descent Parsing**
    - **Predictive Parsing**
  - **Bottom-up parsing**
    - **Shift Reduce Parsing**
    - **Operator Precedence Parsing**
    - **LR Parsers**
      - **SLR Parser**
      - **Canonical LRParser**
      - **LALR Parser**

Recursive –Descent Parser

- Steps are
  - Check whether the Grammar (G) has Left Recursion / Left Factoring
  - If it has Eliminate LR /LF
  - Draw the transition Diagram
  - Minimize the transition diagram

Elimination of left recursion

- We will consider a grammar with productions:

  Left Recursion : $\mathbf{A \rightarrow A\alpha \,/\, \beta}$

  where α and β are sequences of terminals and nonterminals that do not start with A.
- Elimination :

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A / \ \varepsilon$$

Example: An Expression Grammar

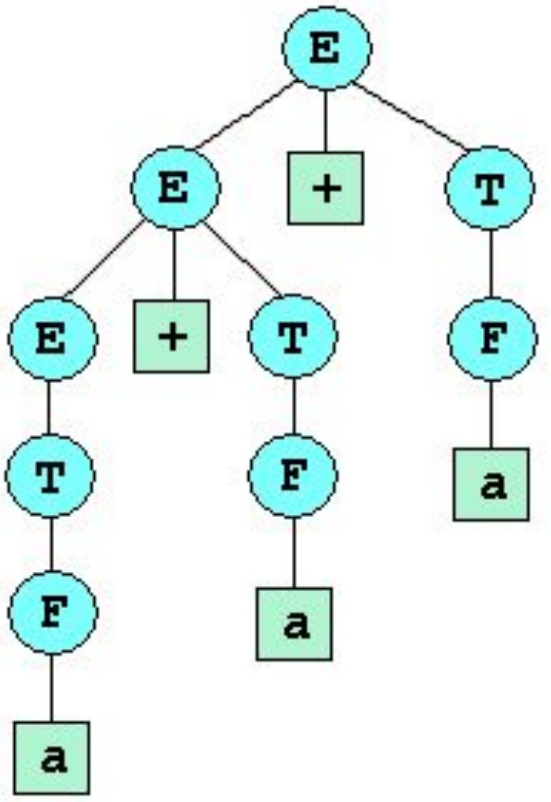
- Let's eliminate left recursion from the grammar below:

  $E \rightarrow E + T \mid T$

  $T \rightarrow T * F \mid F$

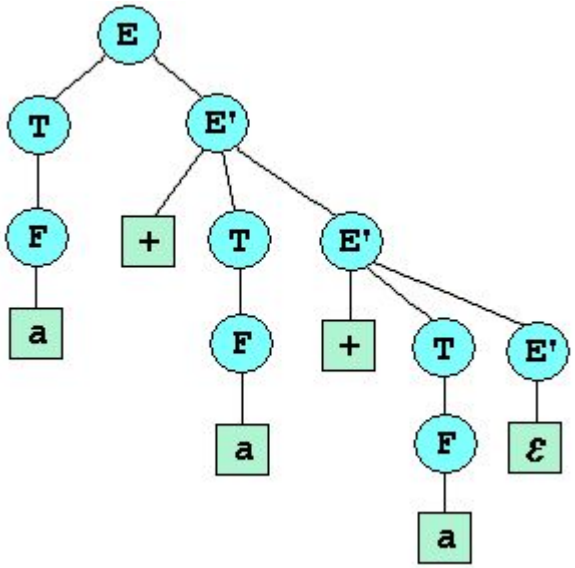
  $F \rightarrow (E) \mid \mathbf{a}$

- The left associativity for + is illustrated by the parse tree for **a** + **a** + **a**:



Example: An Expression Grammar

- Eliminating left recursion gives the grammar below:
  E → TE‘
  E' → +TE' | ε
  T → FT‘
  T' → *FT' | ε
  F → (E) | a
- Note the right associativity of + in the parse tree for **a** + **a** + **a** at right:



Algorithm for Eliminating General Left Recursion

Arrange nonterminals in some order $A_1, A_2, \ldots, A_n$.
for *i* := 1 **to** *n* **do begin**
  for *j* := 1 **to** i - 1 **do begin**
    Replace each production of the form $A_i \rightarrow A_j\beta$ by the productions:

$$A_i \rightarrow \alpha_1\beta \mid \alpha_2\beta \mid \ldots \mid \alpha_k\beta$$

    where $A_j \rightarrow \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_k$ are all the current Aj productions.
  end { for j }

  Remove immediate left recursion from the A_i productions, if necessary
end { for i }

Left Factoring

- In general, if $\mathbf{A \rightarrow \alpha\beta_1 \mid \alpha\beta_2}$ are two A-productions, and the input begins with a nonempty string derived from α, we do not know whether to expand to $\alpha\beta_1$ or to $\alpha\beta_2$.
- Instead, the grammar may be changed. The formal technique is to change the grammar to

$$A \rightarrow \alpha A'$$

$$A' \rightarrow \beta_1 \mid \beta_2$$

Left-Factoring – Example1

$A \rightarrow \underline{a}bB \mid \underline{a}B \mid cdg \mid cdeB \mid cdfB$

$\Downarrow$

$A \rightarrow aA' \mid \underline{cd}g \mid \underline{cd}eB \mid \underline{cd}fB$

$A' \rightarrow bB \mid B$

$\Downarrow$

$A \rightarrow aA' \mid cdA''$

$A' \rightarrow bB \mid B$

$A'' \rightarrow g \mid eB \mid fB$

Left-Factoring – Example2

$A \rightarrow ad \mid a \mid ab \mid abc \mid b$

$\Downarrow$

$A \rightarrow aA' \mid b$

$A' \rightarrow d \mid \varepsilon \mid b \mid bc$

$\Downarrow$

$A \rightarrow aA' \mid b$

$A' \rightarrow d \mid \varepsilon \mid bA''$

$A'' \rightarrow \varepsilon \mid c$

Example

The grammar is as follows

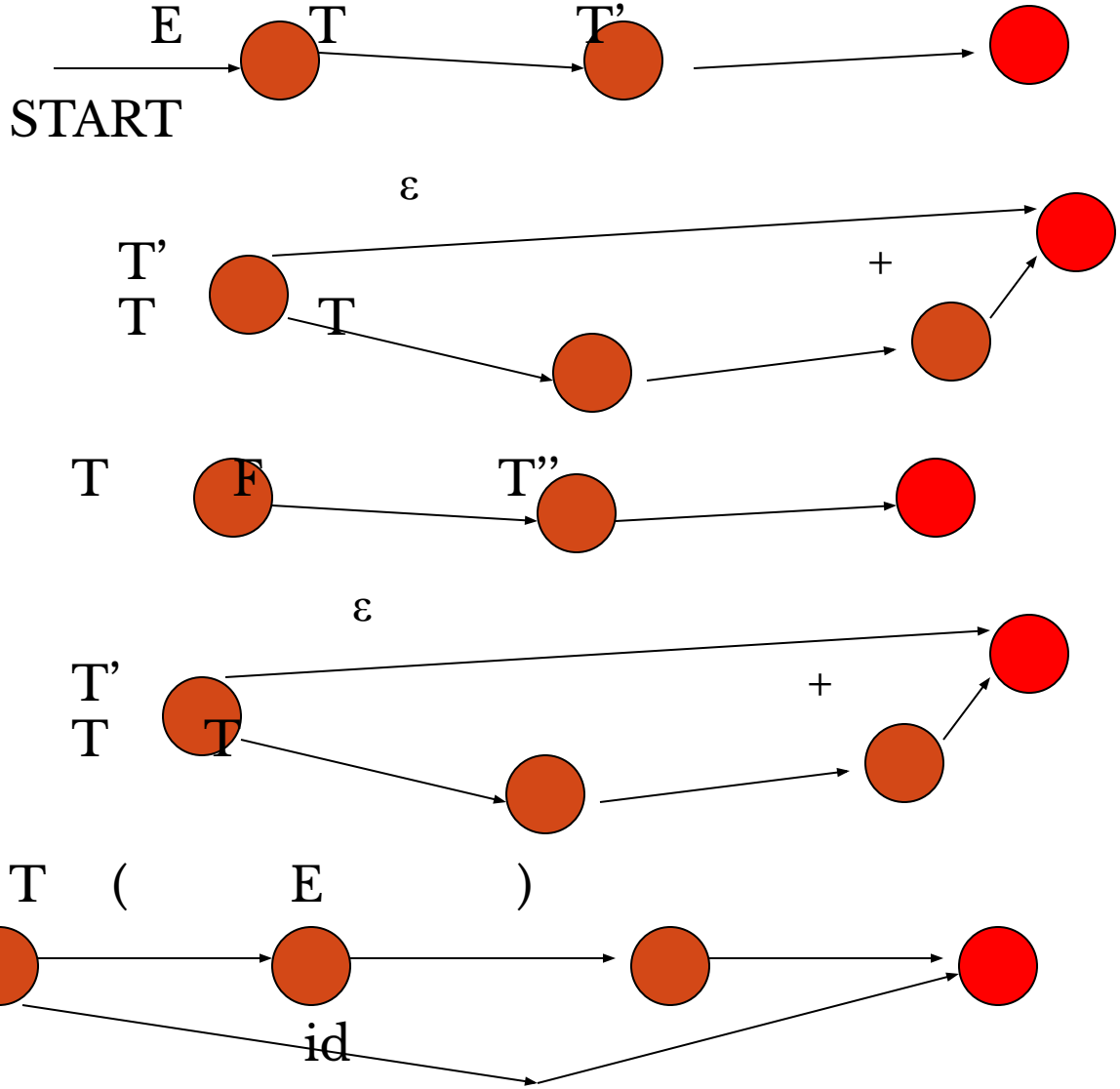
- E -> E + T | T
- T - > T * F | F
- F -> (E) | id

After removing left-recursion , left-factoring

The rules are :

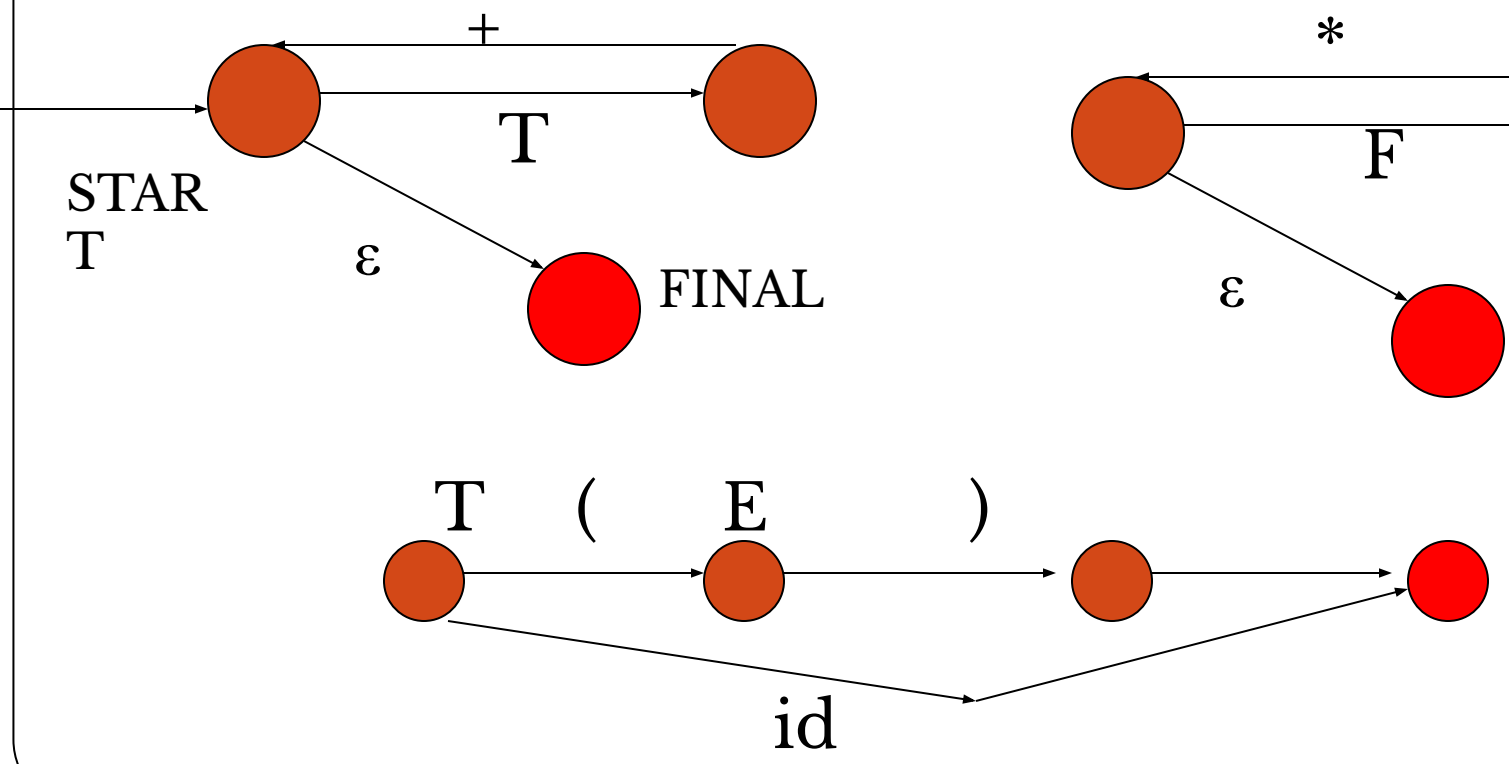
Rules and their transition diagrams

- E->T T'
- T' -> +T T' | ε
- T -> F T''
- T -> *F T'' | ε
- T -> (E) |id



Optimization

After optimization it yields the following DFA like structures:



Definition of First(x)

- Let X be a grammar symbol (a terminal or nonterminal) or ε. Then the set **First(X)** consisting of terminals, and possibly ε, is defined as follows:
- If X is a terminal or ε, then First(X) = {X}.
- If X is a nonterminal, then for each production $X \rightarrow X_1X_2...X_n$, First(X) contains First(X_1) - {ε}.
  - If also for some i < n, all the sets First(X_1),..., First(X_i) contain ε, then First(X) contains First(X_{i+1}) - {ε}.
  - If all the sets First(X_1), First(X_n) contain ε, the

Definition of First(α)

- Now, define First(α) for string $\alpha = X_1X_2...X_n$ (a string of terminals and nonterminals), as follows:
  - First(α) contains First(X_1) - {ε}.
  - For each i = 2,...,n, if First(X_k) contains ε for all k = 1,...i - 1, then First(α) contains First(X_i) - {ε}.
  - Finally, if for all i = 1,...,n, First(X_i) contains ε, then add ε to First(α).

Example: First Sets

- First(a) = {a}, First(ε) = {ε}, First(b) = {b}
- To calculate First(B), look at rules for B:
  - B → b ⇒ First(B) = First(b) = {b}.
- To calculate First(A), look at rules for A:
  - A → a ⇒ First(A) includes {a}
  - A → ε ⇒ First(A) includes {ε}
  - We conclude that First(A) = {a, ε}
- To calculate First(S), look at rule S → AB.
  - First(S) = First(AB), which includes First(A) – {ε} = {a}
  - Since First(A) contains ε, First(AB) also includes First(B) = {b}
  - We conclude that First(S) = {a, b}

S → AB
A → a | ε
B → b

Definition of Follow(A)

- Given a nonterminal A, the set Follow(A), consisting of terminals, and possibly $ (the end of input symbol), is defined as follows:
  1. If A is the start symbol, then $ is in Follow(A).
  2. If there is a production $B \rightarrow \alpha A \gamma$, then First($\gamma$) - {$\varepsilon$} is in Follow(A).
  3. If there is a production $B \rightarrow \alpha A \gamma$ such that ε is in First(γ), then Follow(A) contains Follow(B).

Example: Follow Sets

- By rule 2, if there is a production **B** → **α** **A** **γ**, then First(γ) - {ε} is in Follow(A). Here are examples showing how right hand sides of rules match the pattern above(red is **α**, green is **γ**)
  - X → abYmZ **α** = ab **A** = Y **γ** = mZ
  - X → abYmZ **α** = abYm **A** = Z **γ** = ε
  - B → Axy **α** = ε **A** = A **γ** = xy

Example: Follow Sets

- By rule 3, if there is a production B → α A γ such that ε is in First(γ), then Follow(A) contains Follow(B).
- Examples of right hand sides that match this pattern.
  - A → aX α = a A = X γ = ε
  - A → aXY α = a A = X γ = Y (ε ∈ First(Y))
  - A → Y α = ε A = Y γ = ε

LL(1) Parser

input buffer

- our string to be parsed. We will assume that its end is marked with a special symbol $.

output

- a production rule representing a step of the derivation sequence (left-most derivation) of the string in the input buffer.

stack

- contains the grammar symbols
- at the bottom of the stack, there is a special end marker symbol $.
- initially the stack contains only the symbol $ and the starting symbol S. $S □ initial stack
- when the stack is emptied (ie. only $ left in the stack), the parsing is completed.

parsing table

- a two-dimensional array M[A,a]
- each row is a non-terminal symbol
- each column is a terminal symbol or the special symbol $
- each entry holds a production rule.

LL(1) Parser – Parser Actions

- The symbol at the top of the stack (say X) and the current symbol in the input string (say a) determine the parser action.
- There are four possible parser actions.

1. If X and a are $ □ parser halts (successful completion)
2. If X and a are the same terminal symbol (different from $) □ parser pops X from the stack, and moves the next symbol in the input buffer.
3. If X is a non-terminal □ parser looks at the parsing table entry M[X,a]. If M[X,a] holds a production rule $X \rightarrow Y_1 Y_2 \ldots Y_k$, it pops X from the stack and pushes $Y_k, Y_{k-1}, \ldots, Y_1$ into the stack. The parser also outputs the production rule $X \rightarrow Y_1 Y_2 \ldots Y_k$ to represent a step of the derivation.
4. none of the above □ error
   - all empty entries in the parsing table are errors.
   - If X is a terminal symbol different from a, this is also an error case.

LL(1) Parser – Example1

$S \rightarrow aBa$

$B \rightarrow bB \mid \varepsilon$

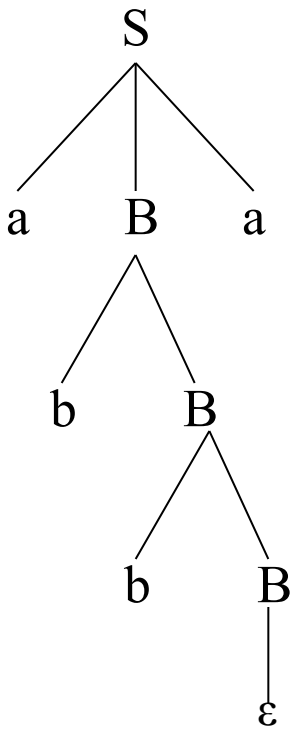
	a	b	$
S	$S \rightarrow aBa$		
B	$B \rightarrow \varepsilon$	$B \rightarrow bB$	

stack	**input**	**output**
$S	abba$	$S \rightarrow aBa$
$aBa	abba$	
$aB	bba$	$B \rightarrow bB$
$aBb	bba$	
$aB	ba$	$B \rightarrow bB$
$aBb	ba$	
$aB	a$	$B \rightarrow \varepsilon$
$a	a$	
$	$	accept, successful completion

LL(1) Parser – Example1 (cont.)

Outputs: S → aBa B → bB B → bB B → ε

Derivation(left-most): S⇒aBa⇒abBa⇒abbBa⇒abba



parse tree

LL(1) Parser – Example2

$E \rightarrow TE'$
$E' \rightarrow +TE' \mid \varepsilon$
$T \rightarrow FT'$
$T' \rightarrow *FT' \mid \varepsilon$
$F \rightarrow (E) \mid id$

	id	**+**	*****	**(**	**)**	**$**
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

LL(1) Parser – Example2

stack	input	output
$E	id+id$	E → TE'
$E'T	id+id$	T → FT'
$E'T'F	id+id$	F → id
$ E'T'id	id+id$	
$ E'T'	+id$	T' → ε
$ E'	+id$	E' → +TE'
$ E'T+	+id$	
$ E'T	id$	T → FT'
$ E'T'F	id$	F → id
$ E'T'id	id$	
$ E'T'	$	T' → ε
$ E'	$	E' → ε
$	$	accept

Constructing LL(1) Parsing Tables

- Two functions are used in the construction of LL(1) parsing tables:
  - FIRST & FOLLOW

- **FIRST(α)** is a set of the terminal symbols which occur as first symbols in strings derived from α where α is any string of grammar symbols.
- if α derives to ε, then ε is also in FIRST(α) .

- **FOLLOW(A)** is the set of the terminals which occur immediately after (follow) the *non-terminal A* in the strings derived from the starting symbol.
  - a terminal a is in FOLLOW(A) if $S \Rightarrow^{*} \alpha A a \beta$
  - $ is in FOLLOW(A) if $S \Rightarrow^{*} \alpha A$

Constructing LL(1) Parsing Table -- Algorithm

- for each production rule $A \rightarrow \alpha$ of a grammar G
  - for each terminal a in FIRST(α)
    - add $A \rightarrow \alpha$ to M[A,a]
  - If ε in FIRST(α)
    - for each terminal a in FOLLOW(A) add $A \rightarrow \alpha$ to M[A,a]
  - If ε in FIRST(α) and $ in FOLLOW(A)
    - add $A \rightarrow \alpha$ to M[A,$]

- All other undefined entries of the parsing table are error entries.

LL(1) Grammars

- A grammar whose parsing table has no multiply-defined entries is said to be LL(1) grammar.

  one input symbol used as a look-head symbol do determine parser action

  LL(1) left most derivation

  input scanned from left to right

- The parsing table of a grammar may contain more than one production rule. In this case, we say that it is not a LL(1) grammar.

A Grammar which is not LL(1)

$S \rightarrow i\ C\ t\ S\ E \mid a$ FOLLOW(S) = { $,e }

$E \rightarrow e\ S \mid \varepsilon$ FOLLOW(E) = { $,e }

$C \rightarrow b$ FOLLOW(C) = { t }

FIRST(iCtSE) = {i}
FIRST(a) = {a}
FIRST(eS) = {e}
FIRST(ε) = {ε}
FIRST(b) = {b}

	a	b	e	i	t	$
S	S → a			S → iCtSE		
E			E → e S E → ε			E → ε
C		C → b				

two production rules for M[E,e]

Problem □ ambiguity

Properties of LL(1) Grammars

- A grammar G is LL(1) if and only if the following conditions hold for two distinctive production rules $A \rightarrow \alpha$ and $A \rightarrow \beta$

1. Both α and β cannot derive strings starting with same terminals.
2. At most one of α and β can derive to ε.
3. If β can derive to ε, then α cannot derive to any string starting with a terminal in FOLLOW(A).

Bottom-Up Parsing

- A **bottom-up parser** creates the parse tree of the given input starting from leaves towards the root.
- A bottom-up parser tries to find the right-most derivation of the given input in the reverse order.

  $S \Rightarrow ... \Rightarrow \omega$ (the right-most derivation of ω)

  $\leftarrow$ (the bottom-up parser finds the right-most derivation in the reverse order)
- Bottom-up parsing is also known as **shift-reduce parsing** because its two main actions are shift and reduce.
  - At each shift action, the current symbol in the input string is pushed to a stack.
  - At each reduction step, the symbols at the top of the stack (this symbol sequence is the right side of a production) will replaced by the non-terminal at the left side of that production.
  - There are also two more actions: accept and error.

Shift-Reduce Parsing

- A shift-reduce parser tries to reduce the given input string into the starting symbol.

  a string □ the starting symbol

  reduced to

- At each reduction step, a substring of the input matching to the right side of a production rule is replaced by the non-terminal at the left side of that production rule.
- If the substring is chosen correctly, the right most derivation of that string is created in the reverse order.

  Rightmost Derivation: $S \Rightarrow_{rm}^{*} \omega$

  Shift-Reduce Parser finds: $\omega \Leftarrow_{rm} \ldots \Leftarrow_{rm} S$

Shift-Reduce Parsing Example

$S \rightarrow aABb$
$A \rightarrow aA \mid a$
$B \rightarrow bB \mid b$

input string: aaabb

aaAbb

aAbb ⇓ reduction

aABb

S

$S \Rightarrow aABb \Rightarrow aAbb \Rightarrow aaAbb \Rightarrow aaabb$

rm rm rm rm

Right Sentential Forms

- How do we know which substring to be replaced at each reduction step?

Handle

- Informally, a **handle** of a string is a substring that matches the right side of a production rule.
  - But not every substring matches the right side of a production rule is handle

- A **handle** of a right sentential form γ ($\equiv \alpha\beta\omega$) is
  a production rule $A \rightarrow \beta$ and a position of γ
  where the string β may be found and replaced by A to produce
  the previous right-sentential form in a rightmost derivation of γ.

$$S \overset{*}{\underset{rm}{\Rightarrow}} \alpha A \omega \underset{rm}{\Rightarrow} \alpha\beta\omega$$

- If the grammar is unambiguous, then every right-sentential form of the grammar has exactly one handle.

Handle Pruning

- A right-most derivation in reverse can be obtained by **handle-pruning**.

  $S = \gamma_0 \underset{rm}{\Rightarrow} \gamma_1 \underset{rm}{\Rightarrow} \gamma_2 \underset{rm}{\Rightarrow} \ldots \underset{rm}{\Rightarrow} \gamma_{n-1} \Rightarrow \gamma_n = \omega$

  input string

- Start from γ_n, find a handle $A_n \rightarrow \beta_n$ in γ_n, and replace β_n in by A_n to get γ_{n-1}.
- Then find a handle $A_{n-1} \rightarrow \beta_{n-1}$ in γ_{n-1}, and replace β_{n-1} in by A_{n-1} to get γ_{n-2}.
- Repeat this, until we reach S.

A Shift-Reduce Parser

$E \rightarrow E+T \mid T$
$T \rightarrow T*F \mid F$
$F \rightarrow (E) \mid id$

Right-Most Derivation of `id+id*id`

$E \Rightarrow E+T \Rightarrow E+T*F \Rightarrow E+T*id \Rightarrow E+F*id$
$\Rightarrow E+id*id \Rightarrow T+id*id \Rightarrow F+id*id \Rightarrow id+id*id$

Right-Most Sentential Form	Reducing Production
`id`+id*id	$F \rightarrow id$
`F`+id*id	$T \rightarrow F$
`T`+id*id	$E \rightarrow T$
E+`id`*id	$F \rightarrow id$
E+`F`*id	$T \rightarrow F$
E+T*`id`	$F \rightarrow id$
E+`T*F`	$T \rightarrow T*F$
`E+T`	$E \rightarrow E+T$
E	

`Handles` are red and underlined in the right-sentential forms.

A Stack Implementation of A Shift-Reduce Parser

- There are four possible actions of a shift-parser action:

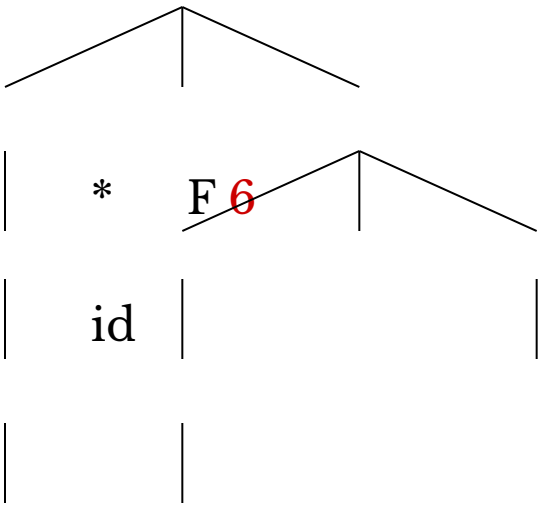
1. **Shift** : The next input symbol is shifted onto the top of the stack.
2. **Reduce**: Replace the handle on the top of the stack by the non-terminal.
3. **Accept**: Successful completion of parsing.
4. **Error**: Parser discovers a syntax error, and calls an error recovery routine.

- Initial stack just contains only the end-marker $.

A Stack Implementation of A Shift-Reduce Parser

Stack	Input	Action
$	id+id*id$	shift
$id	+id*id$	reduce by F → id
$F	+id*id$	reduce by T → F
$T	+id*id$	reduce by E → T
$E	+id*id$	shift
$E+	id*id$	shift
$E+id	*id$	reduce by F → id
$E+F	*id$	reduce by T → F
$E+T	*id$	shift
$E+T*	id$	shift
$E+T*id	$	reduce by F → id
$E+T*F	$	reduce by T → T*F
$E+T	$	reduce by E → E+T
$E	$	accept

Parse Tree



LR Parsers

- The most powerful shift-reduce parsing (yet efficient) is:

  LR(k) parsing.

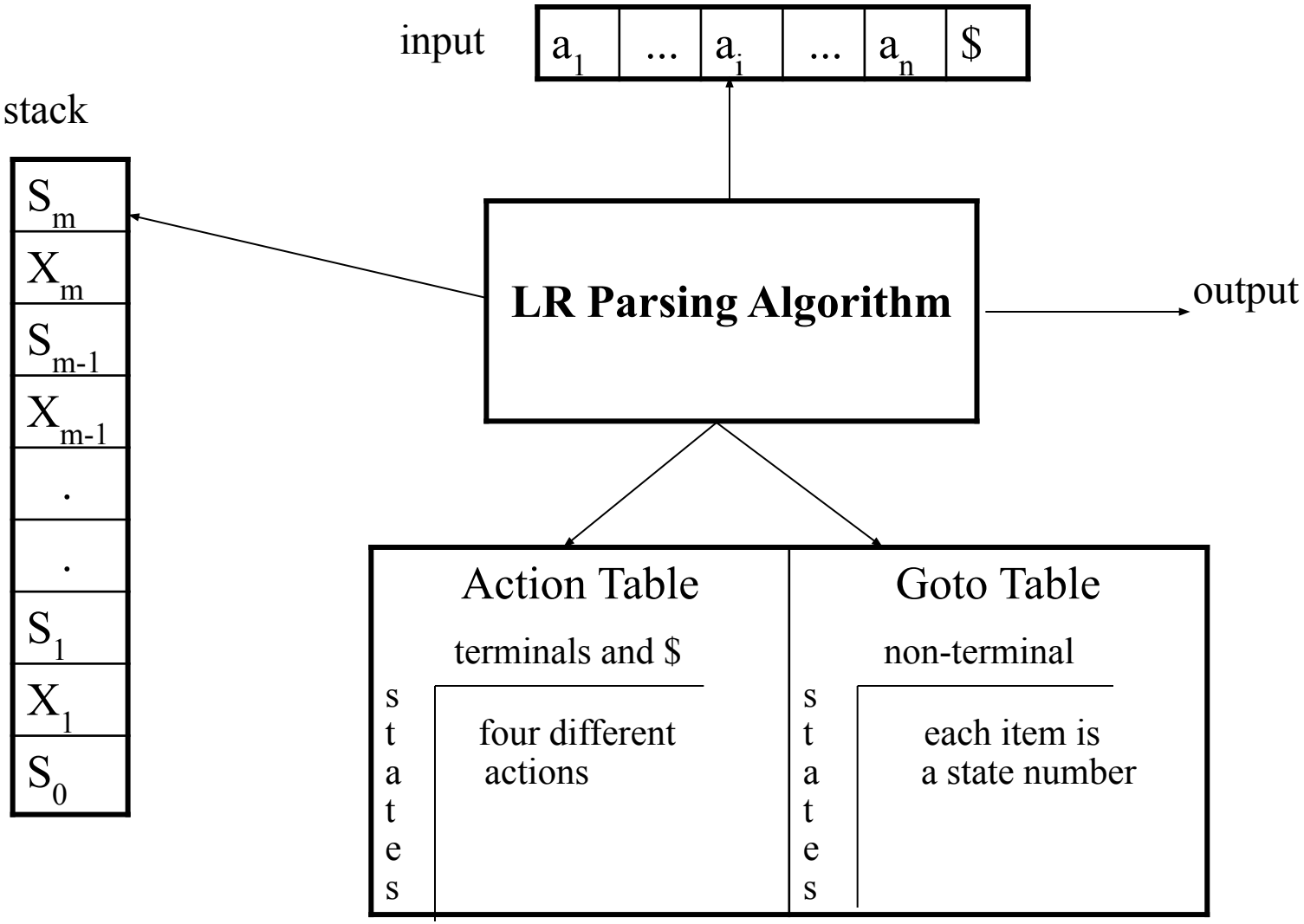
  left to right scanning — right-most derivation — k lookhead (k is omitted ⇒ it is 1)

- LR parsing is attractive because:
  - LR parsing is most general non-backtracking shift-reduce parsing, yet it is still efficient.
  - The class of grammars that can be parsed using LR methods is a proper superset of the class of grammars that can be parsed with predictive parsers.
    LL(1)-Grammars ⊂ LR(1)-Grammars
  - An LR-parser can detect a syntactic error as soon as it is possible to do so a left-to-right scan of the

LR Parsers

- **LR-Parsers**
  - covers wide range of grammars.
  - SLR – simple LR parser
  - LR – most general LR parser
  - LALR – intermediate LR parser (look-head LR parser)
  - SLR, LR and LALR work same (they used the same algorithm), only their parsing tables are different.

LR Parsing Algorithm



A Configuration of LR Parsing Algorithm

- A configuration of a LR parsing is:

  $(S_o X_1 S_1 \ldots X_m S_m, a_i a_{i+1} \ldots a_n \$)$

  Stack Rest of Input

- S_m and a_i decides the parser action by consulting the parsing action table. (*Initial Stack* contains just S_o)

- A configuration of a LR parsing represents the right sentential form:

  $X_1 \ldots X_m a_i a_{i+1} \ldots a_n \$$

Actions of A LR-Parser

1. **shift s** -- shifts the next input symbol and the state **s** onto the stack

   $(S_o X_1 S_1 \ldots X_m S_m, a_i a_{i+1} \ldots a_n \$) \Rightarrow (S_o X_1 S_1 \ldots X_m S_m a_i s, a_{i+1} \ldots a_n \$)$

2. **reduce $A \rightarrow \beta$** (or **rn** where n is a production number)
   - pop $2|\beta|$ (=r) items from the stack;
   - then push **A** and **s** where **$s=goto[s_{m-r},A]$**

   $(S_o X_1 S_1 \ldots X_m S_m, a_i a_{i+1} \ldots a_n \$) \Rightarrow (S_o X_1 S_1 \ldots X_{m-r} S_{m-r} A s, a_i \ldots a_n \$)$

   - Output is the reducing production reduce $A \rightarrow \beta$

3. **Accept** – Parsing successfully completed

4. **Error** -- Parser detected an error (an empty entry in the action table)

Reduce Action

- pop $2|\beta|$ (=r) items from the stack; let us assume that $\beta = Y_1 Y_2 \ldots Y_r$
- then push **A** and **s** where **s=goto[s_{m-r},A]**

  $(S_o X_1 S_1 \ldots X_{m-r} S_{m-r} Y_1 S_{m-r} \ldots Y_r S_m, a_i a_{i+1} \ldots a_n \$)$

  $\Rightarrow (S_o X_1 S_1 \ldots X_{m-r} S_{m-r} A s, a_i \ldots a_n \$)$

- In fact, $Y_1 Y_2 \ldots Y_r$ is a handle.

  $X_1 \ldots X_{m-r} A a_i \ldots a_n \$ \Rightarrow X_1 \ldots X_m Y_1 \ldots Y_r a_i a_{i+1} \ldots a_n \$$

(SLR) Parsing Tables for Expression Grammar

1) E → E+T
2) E → T
3) T → T*F
4) T → F
5) F → (E)
6) F → id

Action Table | Goto Table

state	id	+	*	(	)	$		E	T	F
0	s5			s4				1	2	3
1		s6				acc				
2		r2	s7		r2	r2				
3		r4	r4		r4	r4				
4	s5			s4				8	2	3
5		r6	r6		r6	r6				
6	s5			s4					9	3
7	s5			s4						10
8		s6			s11					
9		r1	s7		r1	r1				
10		r3	r3		r3	r3				
11		r5	r5		r5	r5				

Actions of A (S)LR-Parser -- Example

stack	input	action	output
0	id*id+id$	shift *5*	
0id*5*	*id+id$	reduce by F→id	F→id
0F3	*id+id$	reduce by T→F	T→F
0T2	*id+id$	shift 7	
0T2*7	id+id$	shift *5*	
0T2*7id*5*	+id$	reduce by F→id	F→id
0T2*7F10	+id$	reduce by T→T*F	T→T*F
0T2	+id$	reduce by E→T	E→T
0E1	+id$	shift 6	
0E1+6	id$	shift *5*	
0E1+6id*5*	$	reduce by F→id	F→id
0E1+6F3	$	reduce by T→F	T→F
0E1+6T9	$	reduce by E→E+T	E→E+T
0E1	$	accept	

Constructing SLR Parsing Tables – LR(0) Item

- An **LR(0) item** of a grammar G is a production of G a dot at the some position of the right side.
- Ex: $A \rightarrow aBb$ Possible LR(0) Items: $A \rightarrow \bullet aBb$

  (four different possibility) $A \rightarrow a \bullet Bb$

  $A \rightarrow aB \bullet b$

  $A \rightarrow aBb \bullet$
- Sets of LR(0) items will be the states of action and goto table of the SLR parser.
- A collection of sets of LR(0) items (**the canonical LR(0) collection**) is the basis for constructing SLR parsers.
- *Augmented Grammar*:

  G' is G with a new production rule S'→S where S' is